Understanding How Network Depth Affects Multilayer Perceptron Performance: A Practical Tutorial Using the Wine Classification Dataset

Exploring the Impact of Hidden Layer Depth on Model Accuracy and Generalization

Table of Contents

1. Introduction

The purpose of this tutorial is to explore how changing the depth of a **Multilayer Perceptron (MLP)** influences its performance. We will do this through a small, controlled experiment using the **Wine classification dataset.**

1.1 What you will learn

As you follow this tutorial, you will see:

- What happens when we increase hidden layers
- How depth affects accuracy and generalisation
- When deeper networks help, and when they hurt
- How to evaluate models using visual results and metrics

1.2 Why this technique was chosen

MLPs are ideal for this assignment because they are easy to build, experiment with, and explain step-by-step. A simple change, just by adding or removing layers, gives us clear results to analyse and learn from.

Let's begin the tutorial by understanding what a perceptron and a multilayer perceptron are. **Multilayer Perceptron (MLP)** is one of the most flexible and commonly used models in the neural network. This model is simple enough for beginners to understand, yet powerful enough to model complex relationships in data, which makes this model perfect for learning how architecture choices affect the performance of the model.

1.3 What is a Perceptron?

A perceptron is the simplest kind of artificial neuron that can act as a linear classifier. It takes an input vector, computes a weighted sum plus a bias, then applies a threshold function to decide the output class (often binary). Because of its simplicity, a perceptron can only solve linearly separable tasks (GeeksforGeeks, 2025).

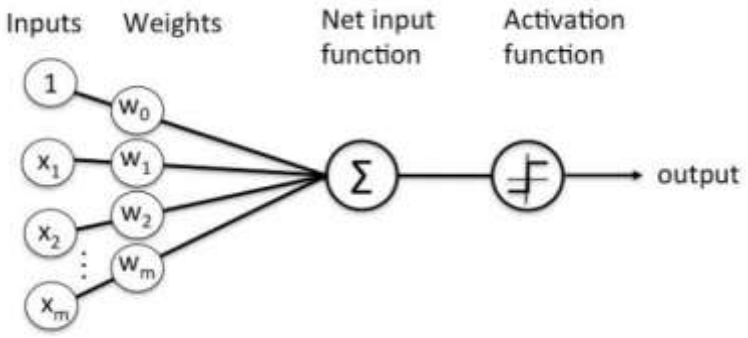

Figure 1: A Perceptron | Source: Medium

1.4 What is MLP?

A **Multilayer Perceptron (MLP)** extends the perceptron by stacking multiple layers of neurons. It is a feed-forward neural network composed of an input layer, one or more hidden layers, and an output layer. In an MLP, neurons are fully connected between successive layers, and hidden layers typically use nonlinear activation functions (e.g., ReLU, sigmoid). This enables the network to learn **non-linear decision boundaries** and represent complex patterns in data (nlunge, 2021).

Because of this difference between a perceptron and a multi-layer perceptron, MLPs can tackle classification and regression tasks that a simple perceptron cannot, and that is how they form the foundation of many neural-network-based machine learning models used today.

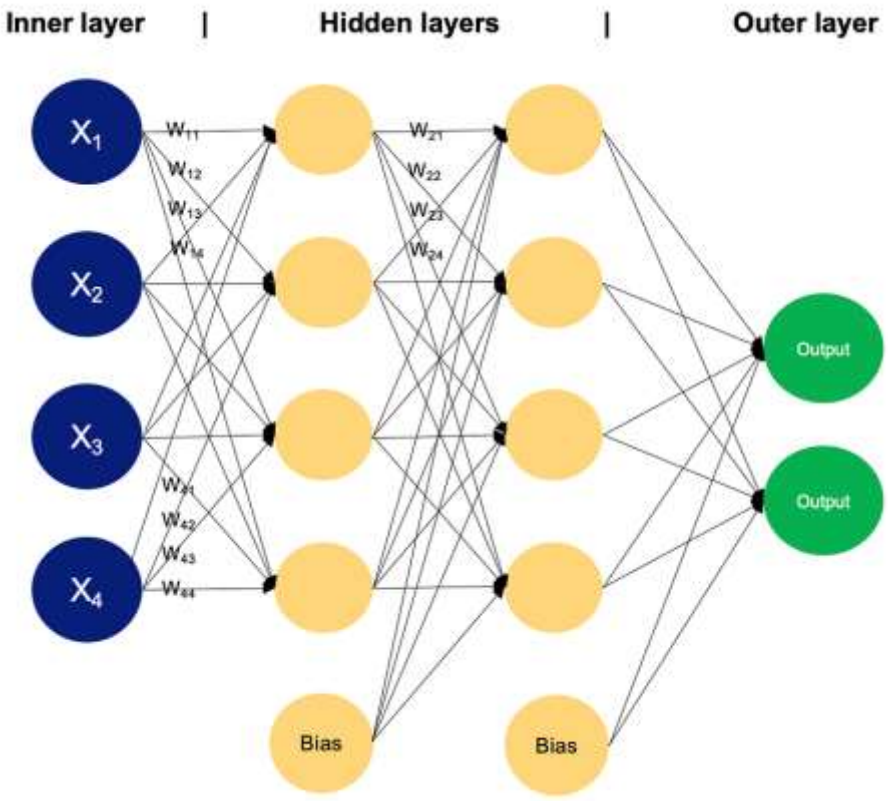

Figure 2: Example of a MLP having two hidden layers | Source: Medium

1.5 Why focus on network depth?

Depth changes everything. Add one hidden layer, the network thinks shallow. Two or three and it starts digging into patterns you do not see on the surface. Hidden layers decide how far the data travels inside the network, how much transformation happens. More layers often mean more steps, more room to twist patterns into something useful. Sometimes it works well, sometimes it fails completely.

A deeper model might learn richer structure, small relations stacked in sequence, one output feeding the next. Yet if you push too far it slows down, training becomes heavy, accuracy even drops. The network becomes too large for the dataset. Too many parameters chasing too little information. It collapses or wanders.

Network depth is a simple lever that has a strong effect. More layers can open the problem up or break the process. This is the main reason for looking at it here.

2. Background Theory

Understanding depth makes more sense once we look at how an MLP learns. Inputs move through the layers, each neuron shifts the values using weights and an activation, and the network tries to guess the right label. It is usually wrong at first. Then it updates the weights and tries again. This cycle continues until the model settles into something that works. This learning process is described in early neural network work by Rumelhart et al. (1986).

2.1 How an MLP learns

Now, let's understand how an MLP learns. During a forward pass, data flows through all layers and produces an output, as seen in Figure 3. The model compares this output with the true label and computes the error. Then the error is pushed backward using backpropagation, as seen in Figure 4, adjusting the weights layer by layer (Goodfellow, Bengio and Courville, 2016). Nonlinear activations such as ReLU or sigmoid are important here because they allow the network to capture patterns that simple linear models cannot (Wikipedia, 2025a).

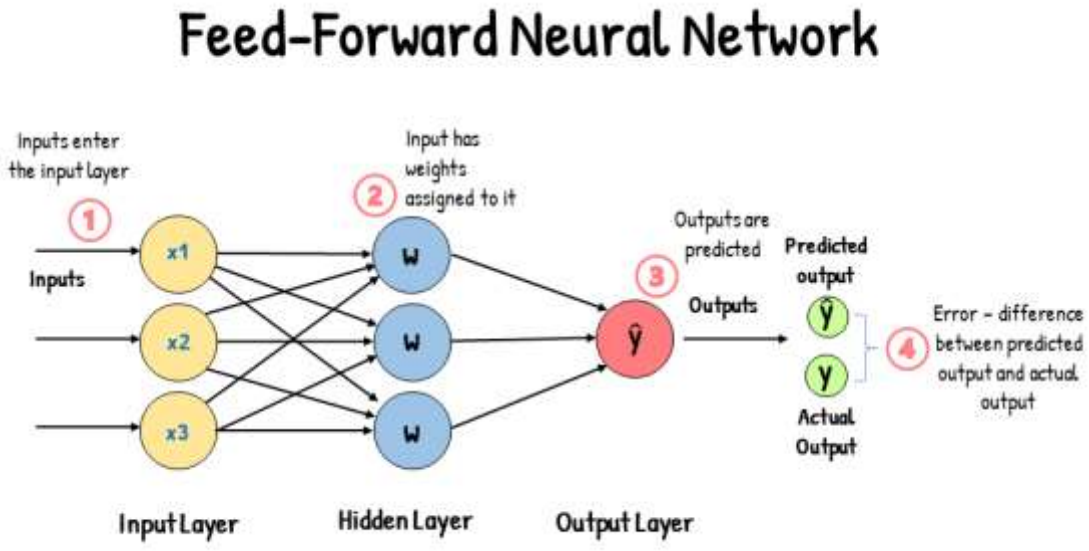

Figure 3: A feed-forward neural network | Source: Analytics Vidhya

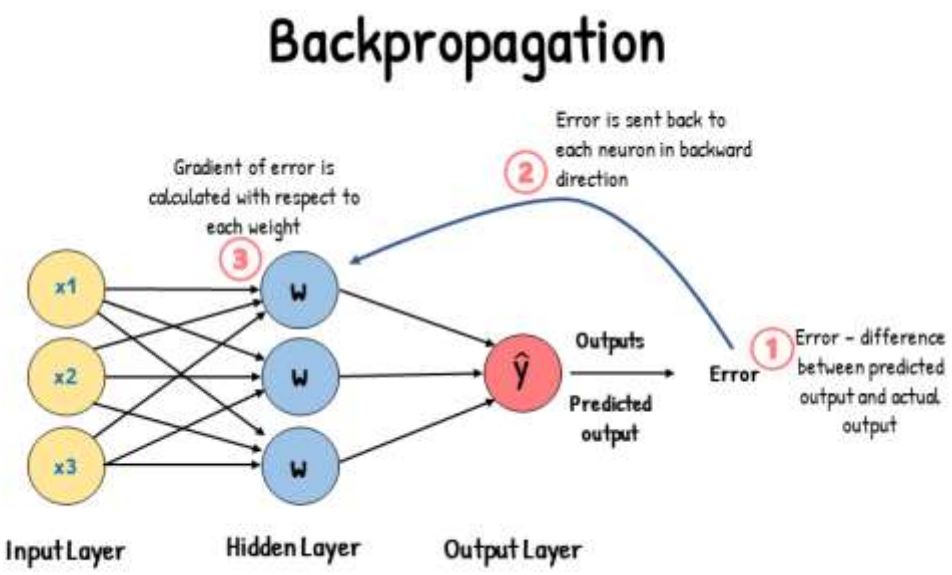

Figure 4: Backpropagation in a neural network | Source: [Analytics Vidhya]

2.2 Why nonlinear layers matter

If all layers were linear, stacking them would collapse into a single linear transformation. Nothing new would be learned. Nonlinear functions Jbreak that limit and make deeper models useful by allowing each layer to transform the data differently (Goodfellow, Bengio and Courville, 2016).

2.3 Depth and representation

Deeper networks build features step by step. Early layers pick up simple patterns. Middle layers combine them. Later layers try to separate classes in a more refined way. This idea appears in many studies on hierarchical representation learning (LeCun, Bengio and Hinton, 2015). Depth gives the model room to express more complex patterns.

2.4 Where depth goes wrong

More layers also add more parameters. With small datasets, the model has too much space to adapt, and it starts memorising examples instead of learning general rules. This is overfitting: the model performs well on training data but fails on unseen inputs. Training slows as well. Gradients may vanish or explode when errors are pushed backward through many layers (Wikipedia, 2025b). Vanishing gradients shrink too much, and the early layers barely learn. Exploding gradients blow up, weights jump wildly, and the model becomes unstable. The benefit does not keep rising. There is usually a point where extra layers add noise instead of insight.

3. Dataset and Experimental Setup

We utilize the Wine Classification dataset to test the hypothesis of the network depth influence on the MLP performance. It consists of 178 samples and 13 measurements of chemicals of each wine which are grouped into three categories. Its small size provides it as a good solution to controlled experiments since deep models can be tested in a short period without intense computation.

StandardScaler is used to scale all the features before training. The chemical properties are highly diversified and include alcohol content, magnesium, and flavonoids, and neural networks have generally been found to train better in a similar scale of features. The training and testing are divided into 80 and 20 percent respectively to be able to evaluate the dataset fairly.

We train four MLP models. All hyperparameters, namely activation function, solver, learning rate, and maximum iterations, remain constant. The only difference between models is the number of hidden layers, keeping the comparison clean. Any change in performance can therefore be attributed to depth.

Table 1: Tested architectures

Model	Hidden Layers
MLP-1	(32)
MLP-2	(64, 32)
MLP-3	(128, 64, 32)
MLP-4	(128, 64, 32, 16, 8)

All the models are trained and evaluated based on the training accuracy, test accuracy and the number of iterations to reach the convergence. Subsequently, a confusion matrix and a classification report are depicted on the effectiveness of the best performing model in classifying the three wine classes.

The aim here is not simply to achieve the highest accuracy. Instead, the experiment focuses on observing how learning behaviour changes as the network becomes deeper. To understand the behaviour of hidden layers. Also, does performance improve? Does training become unstable? The results help answer these questions.

4. Results and Findings

Training the four models revealed clear differences in how depth affects an MLP. Some networks improved with additional layers, while others degraded quickly.

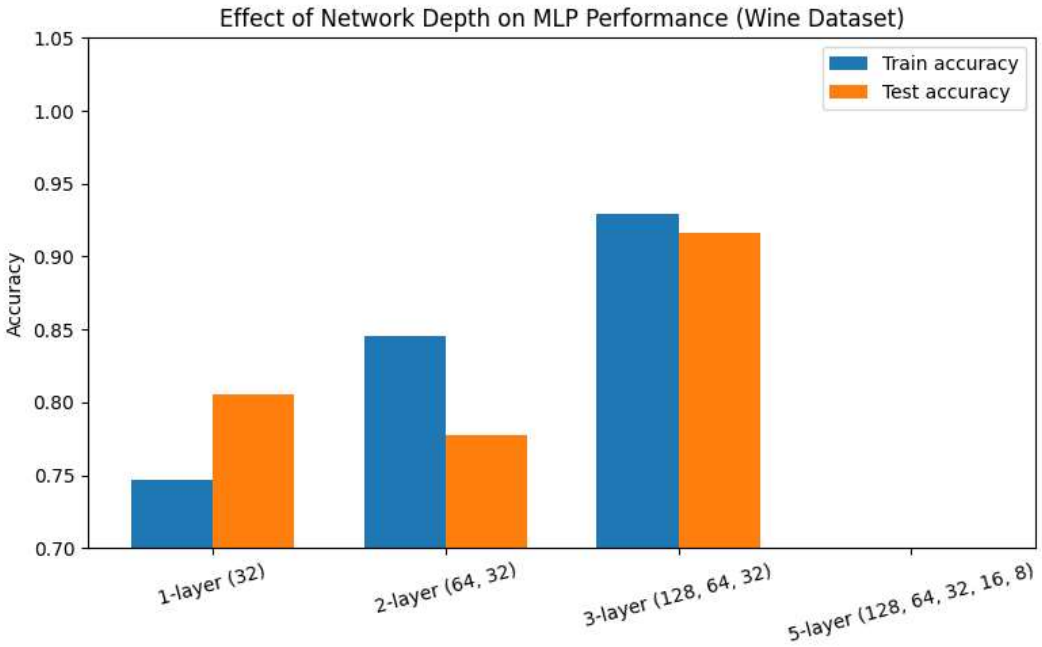

Figure 5: Effect of network depth on MLP performance accuracy.

The 1-layer model was not that bad, as it achieved about mid-70 percent accuracy on the test set. It was able to acquire stable patterns that were not overfit. The 2-layer model improved training accuracy but did not enhance test performance, implying that merely increasing capacity does not necessarily imply improved generalisation using a small dataset.

The most general results were obtained by the 3-layer model. It trained without difficulties, had high training accuracy, and high-test accuracy. This model came with an ideal compromise of depth and stability, and the performance of the model suggests that intermediate depth assists the MLP in mining a more detailed structure of the wine data.

The 5-layer model did not fare well, though. Its training accuracy decreased and test accuracy failed. The size of the dataset could not fit in the network, causing unstable gradients and leading to a hard time. This is an indication that more theoretical insight on deeper networks involves larger datasets and improved regularisation.

This behaviour is well captured in the accuracy table: there is an increase in accuracy between one and three layers, but beyond that, there is a drastic drop in the accuracy of the architecture as the depth becomes excessively large as seen in Figure 6.

	architecture	hidden_layers	train_accuracy	test_accuracy	n_iter
0	1-layer (32)	(32,)	0.746479	0.805556	36
1	2-layer (64, 32)	(64, 32)	0.845070	0.777778	34
2	3-layer (128, 64, 32)	(128, 64, 32)	0.929577	0.916667	31
3	5-layer (128, 64, 32, 16, 8)	(128, 64, 32, 16, 8)	0.401408	0.388889	22

Figure 6: Model architecture comparison of Training and test accuracy by depth

A confusion matrix of the most successful model (the 3-layer network) indicates that the majority of predictions are along the diagonal, and this is a correct prediction. There are a couple of mistakes that are found in the non-diagonal cells.

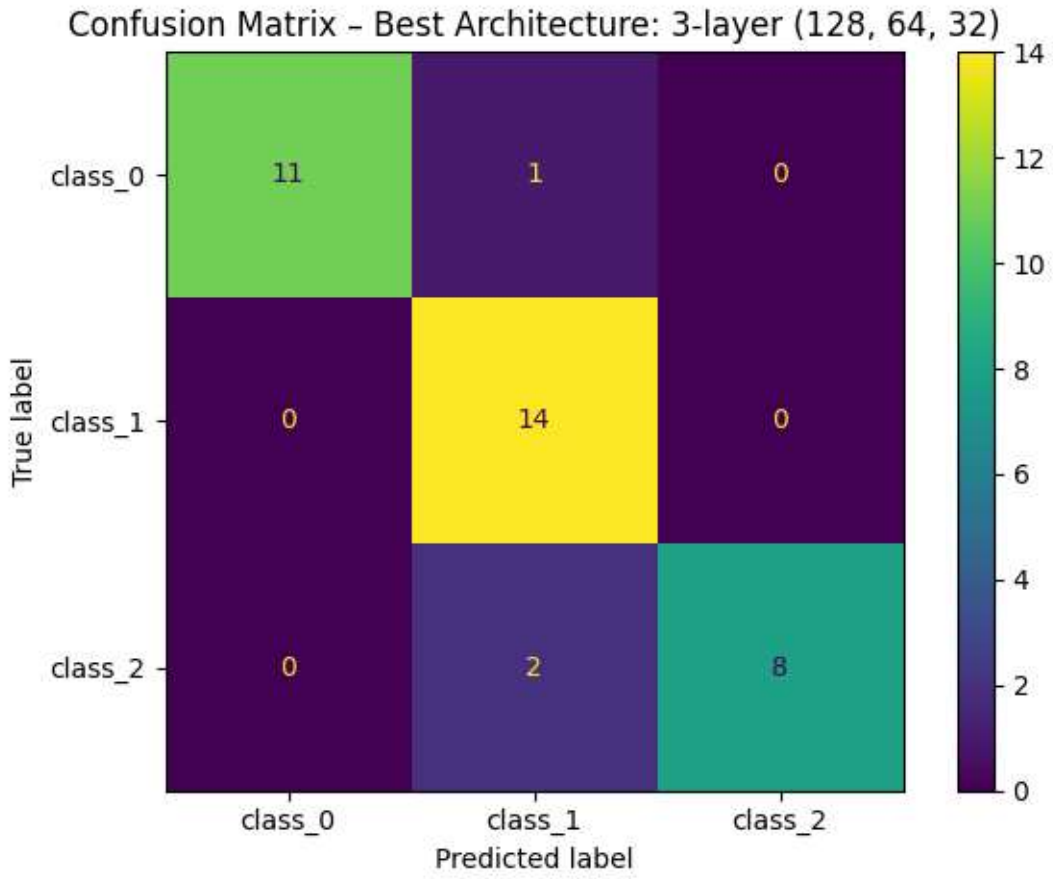

Figure 7: Confusion Matrix for 3-Layer MLP (Best Architecture) Performance.

The classification report supports this by showing high precision and recall across all classes, with only minor variation.

Classification report for best model:

	precision	recall	f1-score	support
class_0	1.00	0.92	0.96	12
class_1	0.82	1.00	0.90	14
class_2	1.00	0.80	0.89	10
accuracy			0.92	36
macro avg	0.94	0.91	0.92	36
weighted avg	0.93	0.92	0.92	36

Figure 8: Classification Report for 3-Layer MLP (Best Architecture) Performance.

Overall, the experiment demonstrates a consistent pattern with depth helps until it hurts. There is a practical sweet spot where the model is deep enough to learn useful representations but not so deep that it becomes unstable. For this dataset, the 3-layer MLP achieves the best balance.

5. Discussion

The findings revealed a simple trend: initially, layers were beneficial, but thereafter they began to complicate things. The model performed well with one hidden layer, though nothing remarkable. With each additional layer, the training accuracy increased since the network was able to learn more complicated patterns. However, that did not last long.

The three-layer model struck the right chord. It did increase training and test accuracy and maintained it. It was not too deep that it began to get unstable or overfit.

After five layers, we put up a front, and it collapsed. The model became too complex with the size of the data set. There was a fall in training accuracy, test accuracy collapsed, and the network was more or less memorising the data rather than general patterns, hence overfitting.

This was due to the fact that deeper networks imply more parameters as well and more parameters demand more data. Additional depth is not useful when the dataset is small, as it only makes the model easier to overfit. Even more than that, deeper networks have gradient problems, in which signal values either decay or explode during training. They are unstable quickly without methods such as using batch norm or initialization.

The moral of the story is that depth is great, but in moderation. Adding more layers does not necessarily help and can worsen the situation, particularly when data is limited.

6. Conclusion

This tutorial has discussed the performance of a Multilayer Perceptron (MLP) with regard to the depth. Comparison of models with various hidden layers showed that increased depth does not necessarily give us better performance. The three-layer MLP has been the most effective and has reached the right balance between complexity and stability.

Adding more depth did aid in learning more complex patterns by the model, but further than that we saw a decline in performance. The dataset got too small to fit into the network, and overfitting began to occur. This experiment demonstrated the depth and dataset size trade-off, and it also shows that increasing the hidden layers or making the network complex does not always help in improving the model learning but it also depends on the dataset complexity.

Important lesson: The Depth of network is important, and so is data volume. Too deep, and the model is not very generalizable. Too superficial, and it is not going to be detailed enough. An easier architecture suffices in the case of small datasets such as the Wine dataset.

Being machine learning practitioners, we will be able to create better models that will save use of time and use of computer power but still get the best results, given that we understand this balance.

The code and notebook for this tutorial can be accessed at the following GitHub repository: GitHub Repository

References

nlunge (2021) *A Deep Architecture: Multi-Layer Perceptron*. Available at: https://medium.com/@nlunge786/a-deep-architecture-multi-layer-perceptron-164bc5ff3842

GeeksforGeeks (2025) 'What is Perceptron | The Simplest Artificial Neural Network'. *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/machine-learning/what-is-perceptron-the-simplest-artificial-neural-network/

Goodfellow, I., Bengio, Y. and Courville, A. (2016) *Deep Learning*. MIT Press.

LeCun, Y., Bengio, Y. and Hinton, G. (2015) 'Deep learning', *Nature*, 521(7553), pp. 436–444.

Rumelhart, D.E., Hinton, G.E. and Williams, R.J. (1986) 'Learning representations by back-propagating errors', *Nature*, 323, pp. 533–536.

Wikipedia (2025a) *Activation function*. Available at: https://en.wikipedia.org/wiki/Activation_function

Wikipedia (2025b) *Vanishing gradient problem*. Available at: https://en.wikipedia.org/wiki/Vanishing_gradient_problem

Analytics Vidhya (2023) 'Gradient Descent vs. Backpropagation: What's the Difference?', *Analytics Vidhya*. Available at: https://www.analyticsvidhya.com/blog/2023/01/gradient-descent-vs-backpropagation-whats-the-difference/